



Fast and accurate Bayesian optimization with pre-trained transformers for constrained engineering problems

Rosen Ting-Ying Yu² · Cyril Picard¹ · Faez Ahmed^{1,2}

Received: 5 April 2024 / Revised: 19 December 2024 / Accepted: 20 February 2025 / Published online: 10 April 2025
© The Author(s) 2025

Abstract

Bayesian Optimization (BO) is a foundational strategy in engineering design optimization for efficiently handling black-box functions with many constraints and expensive evaluations. This paper introduces a novel constraint-handling framework for Bayesian Optimization (BO) using Prior-data Fitted Networks (PFNs), a foundation transformer model. Unlike traditional approaches requiring separate Gaussian Process (GP) models for each constraint, our framework leverages PFN's transformer architecture to evaluate objectives and constraints simultaneously in a single forward pass using in-context learning. Through comprehensive benchmarking across 15 test problems spanning synthetic, structural, and engineering design challenges, we demonstrate an order of magnitude speedup while maintaining or improving solution quality compared to conventional GP-based methods with constrained expected improvement (CEI). Our approach particularly excels at engineering problems by rapidly finding feasible, optimal solutions. This benchmark framework for evaluating new BO algorithms in engineering design will be published at <https://github.com/rosenyu304/BOEngineeringBenchmark>.

Keywords Bayesian optimization · Engineering design optimization · Machine learning · Surrogate-based optimization

1 Introduction

Black-box optimization is a prevalent approach in engineering design optimization, particularly when dealing with problems where the objective function or constraints defining the set are unknown or ambiguous. This method is instrumental in navigating complex design spaces by optimizing solutions without a clear understanding of the underlying functions (Bajaj et al. 2021; Alarie et al. 2021; Tao et al. 2021). Lately, Bayesian optimization (BO) has emerged as a widely adopted black-box optimization tool for its ability to enable evaluations of functions with rapid speed. By leveraging probabilistic models and iteratively

selecting points for evaluation, BO efficiently explores and exploits the design space, making it the forefront of active sampling for optimization (Eriksson and Poloczek 2021; Garnett 2023; Shahriari et al. 2015; Du et al. 2023).

Although BO has emerged as a promising tool for accelerating the search process, the full realization of its potential in engineering design is hindered by the constraint-handling ability of optimization algorithms (Greenhill et al. 2020; Cardoso et al. 2024). A common challenge in design involves identifying products that respect all constraints in the feasible design space. Particularly in structural design optimization, constraints such as cost limitations, regulatory requirements, material constraints, geometric considerations, manufacturing limitations, safety criteria, and ergonomic factors complicate the exploration of design landscapes (Gardner et al. 2017; Baptista and Poloczek 2018; Mathern et al. 2021). Therefore, there is an increase in attention on constraint-handling BO (CBO) algorithms in both engineering and computer science communities (Eriksson and Poloczek 2021; Biswas and Hoyle 2021; Kamrah et al. 2023; Gardner et al. 2014; Ragueneau et al. 2024; Tran et al. 2019, 2022; Ghoreishi and Allaire 2019).

BO's algorithm limitations also come from its commonly used surrogate: the Gaussian Process (GP). By modeling a

Responsible Editor: Axel Schumacher

✉ Rosen Ting-Ying Yu
rosenyu@mit.edu

¹ Department of Mechanical Engineering, Massachusetts Institute of Technology, 77 Massachusetts Ave, Cambridge, MA 02139, USA

² Center for Computational Science and Engineering, Massachusetts Institute of Technology, 77 Massachusetts Ave, Cambridge, MA, USA

function with the mean and the kernel (covariance) function, GP suffers from cubic time complexity $\mathcal{O}(n^3)$ of n training points, leading to scalability issues and runtime concerns (Liu et al. 2020; Gilboa et al. 2013). The need to repeatedly refit and infer in GP-based BO exacerbates the computational time demands. Moreover, the conventional approach for handling constraints in BO, the constrained expected improvement (CEI) method (Gelbart et al. 2014), requires a separate GP for each constraint, making the time scale with the number of constraints. The cubic time complexity, GP scalability issues, and need for multiple GPs to handle constraints make traditional BO methods particularly limiting when dealing with optimization problems with many constraints, as the computational burden grows substantially with increased dimensionality and number of constraints. Therefore, studies have focused on accelerating GP-based BO and improving GP's scalability GP (Cunningham et al. 2008; Foreman-Mackey et al. 2017; Klein et al. 2017; Martinez-Cantin 2018; Pleiss et al. 2020).

To address the runtime limitations, Müller et al. (2023) have proposed a novel zero-training transformer framework known as Prior-data Fitted Networks for BO (PFNs4BO), which bypasses the fitting phase by leveraging a pre-trained model. The center of this framework is the Prior-data Fitted Network (PFN), a transformer architecture meta-trained on a vast dataset of synthetically generated priors. Unlike traditional hybrid approaches merging machine learning models with optimization techniques that require retraining for different problems, PFN is a problem-agnostic pre-trained model capable of solving diverse problems without additional training (Khan et al. 2020, 2022; Ali et al. 2020; Zhang et al. 2021). This pre-trained nature also enables PFN to produce posterior predictive distributions without additional training, offering a promising solution for BO by speeding up the optimization process by an order of magnitude. Although PFNs4BO has demonstrated a substantial speed improvement, its applicability has been confined to single-objective optimization problems without constraints.

Despite the computer science and engineering community's numerous novel BO algorithms, there is a scarcity of open-source benchmark test suites, particularly for constrained engineering optimization problems. Various studies assess BO algorithms on distinct problem scenarios, such as material discovery (Liang et al. 2021) or chemical engineering experiments (Shields et al. 2021), making it challenging to compare methods across a broad range of engineering applications. Common optimization benchmark sets include COCO (Hansen et al. 2021), special sessions of competition in optimization at the Congress on Evolutionary Computation (IEEE CEC) and the Genetic and Evolutionary Computation Conference (GECCO), and Pymoo (Blank and Deb 2020). However, most of these benchmarks consist of synthetic numerical problems that

may not accurately represent the challenges of engineering design problems (Picard and Schiffmann 2021). Of the published engineering design problems, few have their code publicly available while others are not easily interfaced with state-of-the-art BO libraries, limiting their accessibility and utility (Gandomi et al. 2011; Yang and Hossein Gandomi 2012; Eriksson and Poloczek 2021; Jetton et al. 2024). Additionally, most constrained engineering optimization problems in the literature are tested with Genetic Algorithms (GA; Gandomi et al. 2011; Yang and Hossein Gandomi 2012), making it difficult for researchers interested in testing with BO to find relevant studies. Overall, this results in a gap in the availability of a comprehensive engineering benchmark.

The focus of our study is to evaluate the performance of the recently published Prior-data Fitted Network (PFN)-based Bayesian Optimization (BO) in solving constrained engineering design optimization problems. We aim to demonstrate the effectiveness and potential superiority of general-purpose models that eliminate the need for fitting at each BO iteration, compared to traditional GP-based BO using common constraint-handling approaches. This could expand BO's applicability to time-sensitive engineering tasks such as interactive experiment design assistance or robotics control optimization. Our contributions include:

- (1) *A pre-trained transformer-based CBO algorithm* we developed a constrained-handling PFN-based algorithm utilizing the Constrained Expected Improvement (CEI) acquisition function. Our PFN-CEI framework reduces the computational burden typically associated with GP-CEI by utilizing only one pre-trained surrogate model, rather than requiring multiple surrogates for objectives and constraints. The PFN model's transformer architecture enables the simultaneous evaluation of objectives and constraints in a single forward pass. As a pre-trained model, PFN eliminates the need for iterative training, further enhancing its efficiency. PFN-CEI exhibits superior optimization performance compared to all other tested methods and is faster than GP-based CEI.
- (2) *Speed and performance comparison of CBO methods* we present three constraint-handling methods and two surrogate modeling approaches, evaluating their speed and optimization performance. We highlight that using PFN as the surrogate for CBO can achieve a **tenfold increase in speed compared to GP** and outperform GP with a superior anytime performance and feasibility rate. Additionally, we provide a reflective discussion on the potential of PFN-based BO as a fast optimization algorithm.
- (3) *Open-source code and enhanced benchmark tools* we present a set of 15 test problems for benchmarking BO

algorithms, featuring engineering problems with many constraints from the literature. To foster collaborative progress, we make our constrained test problem set and corresponding Python codebase available at <https://github.com/rosenyu304/BOEngineeringBenchmark>, encouraging other researchers to build upon and advance engineering Bayesian optimization.

In this work, the background of engineering design optimization problems, Bayesian optimization with constraint-handling methods, and PFN's application on BO are described in Sect. 2. Section 3 defines the constraint-handling BO algorithms of interest, the test problem set for benchmark, and the evaluation methods for CBO algorithms. The results of algorithm runtime and optimization performance are presented in Sect. 4. Finally, Sect. 5 discusses the overall performance of the tested CBO algorithms.

2 Background

In this section, we introduce the common design optimization problems and methods, detail the Bayesian optimization algorithm, and highlight how the PFN-based BO method differs from traditional BO methods.

2.1 Bayesian optimization for design optimization problems

Engineering design optimization problems are often formulated as one optimization objective subjected to many inequality constraints. The mathematical representation of such problems can be written in this form:

$$\begin{aligned} & \min_{x \in \mathbb{R}^d} f(x) \\ & \text{s.t. } g_i(x) \leq 0, \quad i \in [1, G], \end{aligned} \quad (1)$$

where x is the design variable with dimension d , $f(x)$ is the objective function, and $g_i(x)$ is the constraint with G as the numbers of constraints.

In general, finding the optimum of an optimization problem is non-trivial. The difficulty primarily stems from the ambiguity of objectives and constraints, coupled with the complexity of their evaluation. In engineering design, evaluating objective functions and constraints often involves physical experiments or complex simulations that are time-consuming and expensive. For instance, based on Ford Motor Company, conducting a car crash simulation may require 36 to 160 h per experiment (Wang and Shan 2006). Researchers must then update their dataset, run the optimization algorithm—a process that itself takes time—and repeat this cycle numerous times. This leads to slow data collection and resulting in a dataset too small for accurate

predictions using machine learning-based surrogate models. Thus, BO, with its efficiency in data usage and ability to incorporate prior knowledge for surrogate-based global optimization, emerges as an ideal solution for engineering design optimization tasks.

BO is an active-learning algorithm designed for black-box optimization that iteratively improves performance through exploitation and exploration (Garnett 2023). This process begins with a relatively small set of initial samples, typically ranging from 20 to 50 samples, depending on the complexity and dimensions of the problem at hand. BO employs a probabilistic surrogate model, commonly a Gaussian Process, to form a posterior belief about the design space. Utilizing this posterior, an acquisition function is then applied to determine the most promising next candidate, the one that is likely to be the optimum within the given space. Several acquisition functions commonly used in the literature are the probability of improvement (PI), expected improvement (EI), entropy search (ES), and upper confidence bound (UCB) (Garnett 2015). Algorithm 1 demonstrates a general framework of the BO algorithm.

Algorithm 1 Bayesian optimization (BO)

Require: x_0 initial samples, $f(\cdot)$ the objective function, N_{iter} the iterations set for the algorithm to run, NEXTEVAL an algorithm using: Model($\cdot$) a surrogate and α an acquisition function for determining the next candidate for searching

Ensure: The optimal solution x_{opt}

```

1: function BO( $x_0, N_{\text{iter}}, f(x), \alpha$ )
2:   Provide or perform initial sampling of  $x_0$ 
3:    $X \leftarrow x_0$ 
4:   for  $N_{\text{iter}}$  iterations do
5:      $D \leftarrow \{X, f(X)\}$ 
6:      $x_{\text{next}} \leftarrow \text{NEXTEVAL}(\text{Model}(\cdot), D, \alpha)$ 
7:      $X \leftarrow X \cup x_{\text{next}}$   $\triangleright$  Append  $x_{\text{next}}$  to  $X$ 
8:   end for
9:   return  $x_{\text{opt}} \leftarrow \arg \max(f(X))$ 
10: end function
```

2.2 Constraint-handling Bayesian optimization (CBO)

Constraint-handling Bayesian optimization (CBO) has emerged as a key area in design optimization, addressing engineering limitations such as cost, ergonomics, safety, and regulatory standards. This study focuses on solving single-objective optimization problems with G constraints. Two main

categories of constraint-handling approaches are typically employed for this type of problem: objective transformation and acquisition function modification.

2.2.1 Objective transformation

One common approach for constraint handling involves penalizing the objective value of infeasible data or increasing the objective value of feasible data through an objective function transformation. The penalty function (PF) is a widely-used method that alters the objective values of infeasible data (Fletcher 1975) by introducing a penalty term. Following this transformation, the BO algorithm is applied to the modified unconstrained optimization problem, aiming to minimize f_{PF} . In constrained optimization problems, the quadratic form of the penalty function is often employed, as illustrated in the following equation:

$$f_{\text{PF}}(x) = f(x) + \rho \sum_{i=1}^G \max(0, g_i(x))^2. \quad (2)$$

Equation (2) shows that, given one objective and G constraints to be optimized, $f_{\text{PF}}(x)$ is the penalty transformed objective that is calculated using the objective function $f(x)$, the constraint functions $g_i(x)$, $i \in [1, G]$, and ρ is the penalty factor. The selection of the penalty factor value varies across different studies. In this paper, we initialize $\rho = 1$ and multiply ρ by 1.5 if the algorithm fails to identify an improved optimal value after five iterations (Jetton et al. 2024). However, a limitation of this method is its difficulty in implementation when the constraints cannot be represented analytically by numerical equations.

2.2.2 Acquisition function modification

As evaluating black-box constraints alongside the objective function through analytical transformation can be challenging, one strategy is to treat constraint functions as feasibility objectives. This requires surrogate modeling and their inclusion in the acquisition function calculation. Consequently, the constraint-handling Bayesian optimization (CBO) process employs $1 + G$ surrogate models for modeling the objective function with G constraints. These surrogate models for constraints, known as feasibility models, determine the probability of feasibility P_{feas} as shown in Eq. (3).

One of the most popular objective acquisition functions, expected improvement [EI, see Eq. (4)], has been adapted for constrained optimization. Gelbart et al. (2014) proposed the constrained EI (CEI) acquisition function as the sum of EI and P_{feas} for each constraint shown in Eq. (5).

$$P_{\text{feas}} = \Phi\left(\frac{-\hat{g}(x)}{\sigma_{g(x)}}\right), \quad (3)$$

$$\alpha_{\text{EI}} = (f_* - \hat{f}(x)) \Phi\left(\frac{f_* - \hat{f}(x)}{\sigma_{f(x)}}\right) + \sigma_{f(x)} \mathcal{N}\left(\frac{f_* - \hat{f}(x)}{\sigma_{f(x)}}\right), \quad (4)$$

$$\alpha_{\text{CEI}} = \alpha_{\text{EI}} \prod_{i=1}^G P_{\text{feas},i}, \quad (5)$$

where Φ is the Gaussian cumulative distribution function (CDF), $\hat{g}(x)$ is the mean value of g at point x , $\sigma_{g(x)}$ is the standard deviation of $g(x)$, f_* is the minimum (optimum) observed value up until the current iteration, $\hat{f}(x)$ is the mean value of f at point x , $\sigma_{f(x)}$ is the standard deviation of $f(x)$, and $\mathcal{N}$ is the Gaussian (Normal) distribution.

One limitation of CEI is that the combined probability of feasibility will be close to zero at the edge of the constraint regions. Therefore, a modified version of the CEI algorithm has been proposed to increase the chance of selecting solutions near the constraint boundaries (Bagheri et al. 2017):

$$\alpha_{\text{CEI}+} = \alpha_{\text{EI}} \prod_{i=1}^G \min(1, 2P_{\text{feas},i}). \quad (6)$$

2.3 Prior-data fitted network and its application on Bayesian optimization

A Prior-Data Fitted Network (PFN) is a transformer framework trained to perform Bayesian inference (Müller et al. 2021). Just as large language models (LLMs) like ChatGPT learn to predict text from context (the prompt), PFNs learn to predict posterior distributions from observed data. The key advantage is its ability to generalize to new inputs without retraining—a limitation that has long challenged traditional Bayesian inference surrogate models that requires refitting for new datasets.

The PFN model undergoes a single meta-training phase, leveraging more than nine million synthetic datasets (Müller et al. 2021). These datasets integrate prior-posterior pairs from randomly initialized Gaussian Processes (GPs) with the well-engineered GP priors instantiated from Heteroscedastic Evolutionary Bayesian Optimization (HEBO; Cowen-Rivers et al. 2022). Through learning to predict the masked test data conditioned on training data, PFN learns to perform Bayesian inference. The one-time training on a diverse and large amount of datasets allows PFN to do in-context learning at inference time. The in-context learning ability enables PFN

to process new query data $\hat{x}$ alongside the observed example dataset $D = \{(x_1, y_1), \dots, (x_k, y_k)\}$ in a single pass through its fixed-parameter model, similar to how humans draw from past experiences to tackle new but related tasks.

PFN model's transformer architecture enables its in-context learning ability. The model first uses a linear layer to project both inputs x (feature data) and y (labels) from the dataset to a high-dimensional vector space (512 dimensions), encoding the dataset-level representation. Unlike LLMs where position matters for text understanding, here the data points are explicitly made position-invariant since the order of examples in a dataset shouldn't matter for Bayesian inference. The encoded representation embeddings are then passed through the transformer encoder layers with the multihead attention mechanism. Within each attention layer, the model maps dataset example embeddings (encoded x , y pairs) into queries, keys, and values. The attention mechanism then computes similarities between these representations, determining how strongly each input example and query point attends to the relevant data points in the dataset. Specifically, input examples can attend to other input examples to capture dataset-wide patterns, while query points can only attend to input examples to make predictions based on the observed data. During training, the model samples tasks from a prior distribution, draws data points and labels, masks one label, and learns to predict it based on the remaining points. The training process fixes a total number $N = n + m$ (where n is input size and m is queries) and samples n from a distribution to allow the model to handle different dataset sizes. At inference time, it takes a real dataset and test point as input and outputs a prediction distribution in a single forward pass (Müller et al. 2021).

With PFN's ability to do Bayesian inference without retraining, PFN is integrated into the Bayesian optimization framework, named as PFNs4BO, as the surrogate to do efficient BO (Müller et al. 2023). By passing in the input observed data (D) and the query points ($\hat{x}$) sampled from the unknown search space, PFN predicts the posterior distributions $p(\hat{y} | \hat{x}, D)$ of the query points without retraining. With the PFN predicted posterior, acquisition values of the query points ($\alpha(\hat{x})$) can be calculated, and the sample with the highest acquisition value will be evaluated and appended to the observed data for the next search in the BO iterations. The key advantage of PFN-based BO over traditional GP-based approaches lies in its efficiency. While GP-based BO requires refitting at every iteration and relies on an acquisition function optimizer, PFN-based BO eliminates both requirements through its in-context learning ability.

The current released PFN model on the PFNs4BO GitHub repository¹ is limited to 18 design variables and

requires retraining for higher dimensions. While it supports three basic acquisition functions (EI, PI, and UCB), it can only perform unconstrained single-objective optimization. Our study expands this framework's practical applications by introducing a constraint-handling acquisition function that leverages PFN's transformer architecture for parallel data processing and optimization.

Algorithm 2 Surrogate modeling and acquisition function

Require: Model($\cdot$) that returns a posterior distribution of the input data D , α an acquisition function, $\mathcal{X}$ search space

Ensure: x_{next} the next search point

```

1: function NEXTEVAL(Model( $\cdot$ ),  $D$ ,  $\alpha$ )
2:   if Model is GP then
3:      $\text{GP}(D) \leftarrow$  Fitting GP with  $D$ 
4:      $x_{\text{next}} \leftarrow \arg \max_{\hat{x} \in \mathcal{X}} \alpha(\hat{x}, \text{GP}(D))$ 
5:                                      $\triangleright$  optimizing  $\alpha$ 
6:   end if
7:
8:   if Model is PFN then
9:      $x_{\text{next}} \leftarrow \arg \max_{\hat{x} \in \mathcal{X}} \alpha(\hat{x}, \text{PFN}_{\theta}(\cdot | D))$ 
10:  end if
11:  return  $x_{\text{next}}$ 
12: end function

```

3 PFN-based CBO frameworks

Here we propose three PFN-based CBO frameworks leveraging transformer architecture to handle constraints. While traditional GP-based methods require separate models for each constraint, our approach leverages the parallel processing capabilities of transformer architectures to handle multiple constraints efficiently. We present three different constraint-handling approaches: PFN-Pen (PFN with penalty function), PFN-CEI (PFN with constrained EI), and PFN-CEI+ (PFN with modified constrained EI). Figure 1 visualizes the difference between GP-based and PFN-based BO.

3.1 PFN-Pen

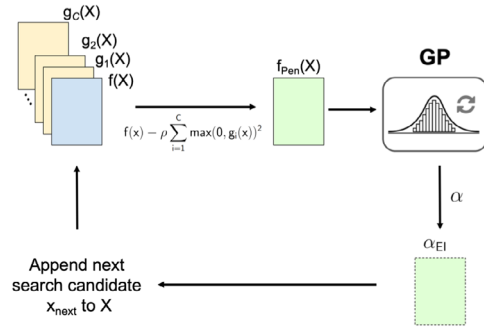
Using the penalty transform method discussed in Sect. 2.2.1, PFN-Pen performs Bayesian optimization on the transformed objective $f_{\text{PF}}(X)$ and outputs the posterior for acquisition function α_{EI} . For calculating $f_{\text{PF}}(X)$, we initialize the penalty factor $\rho = 1$ and multiply ρ by 1.5 when the algorithm fails to identify an improved optimal value after five iterations (Jetton et al. 2023). PFN's pre-trained nature

¹ <https://github.com/automl/PFNs4BO/>.

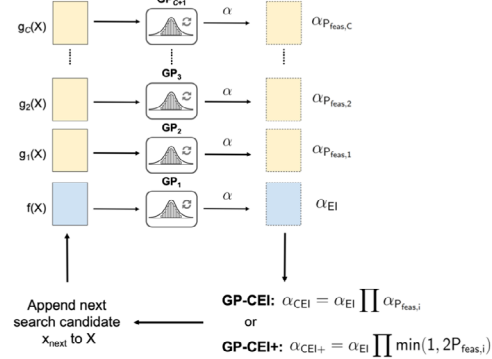
Constrained BO algorithm overview:

 : Iteratively retrain
 : No retrain

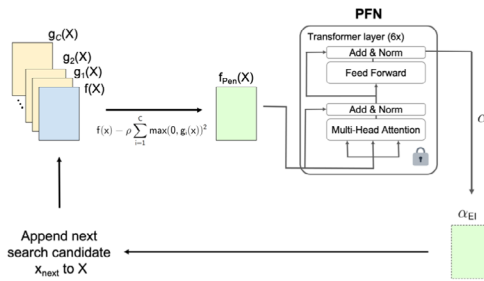
a) GP-PEN:



b) GP-CEI/CEI+:



c) PFN-PEN:



d) PFN-CEI/CEI+:

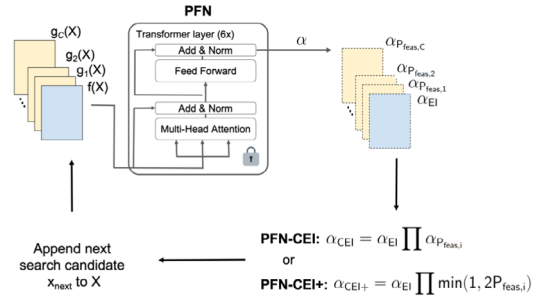


Fig. 1 Comparison of constrained Bayesian optimization (BO) approaches: **a** GP-Pen: traditional GP-based BO with penalty function transformation, **b** GP-CEI/CEI+: given an objective and G constraints, GP-CEI requires $G + 1$ GPs to perform one search iteration for BO, with each GP needing to be fit and updated in every iteration, **c** PFN-Pen: PFN-based BO with penalty function transformation, and

d PFN-CEI/CEI+: only one PFN is needed for optimizing an objective and G constraints, and no fitting of PFN occurs during BO since it is a pre-trained model. PFN's transformer architecture allows the EI of the objective and P_{feas} of the constraints to be solved in parallel in one pass without retrain

eliminates the need for repeated model fitting at each iteration when using GP.

$$\alpha_{EI}(\text{PFN}_{\theta}(\cdot|\{X, f_{PF}(X)\})). \quad (7)$$

3.2 PFN-CEI

To implement CEI constraint-handling method stated in Sect. 2.2.2, the calculation of α_{EI} and P_{feas} of each constraint function are required. In contrast to the GP-based approach, which requires a separate GP for each objective and constraint, a PFN can solve for the acquisition values for α_{EI} and P_{feas} in one forward pass using a single surrogate. Leveraging the transformer architecture of PFN, which supports batch processing, we develop a method to simultaneously solve objectives and constraints with a single model. This architectural advantage significantly reduces computational overhead, especially for problems with many constraints. Figure 1's b, d highlight the differences between GP-CEI and PFN-CEI.

$$\alpha_{CEI}(\text{PFN}_{\theta}(\cdot|\{X, f(X)\})). \quad (8)$$

3.3 PFN-CEI+

This method adds the modified CEI+ threshold mentioned in Sect. 2.2.2 to the PFN-CEI algorithm to handle constraint boundaries.

$$\alpha_{CEI+}(\text{PFN}_{\theta}(\cdot|\{X, f(X)\})). \quad (9)$$

4 Experiments

This section describes our constrained optimization problems, on which constrain-handling Bayesian Optimization algorithms are tested, and the evaluation metrics.

4.1 CBO algorithms

In this study, we focus on benchmarking the performance of three proposed PFN-based CBO algorithms as highlighted in Sect. 3 with the current state-of-the-art GP-based BO using BoTorch library (Balandat et al. 2020), an open-source Bayesian optimization tool based on PyTorch. To make a fair comparison between algorithms using two different surrogates, we implement the same constraint-handling methods on GP and formulate three GP-based CBO algorithms: GP-Pen, GP-CEI, and GP-CEI+. A detailed visualization of all six CBO algorithms tested in this study is detailed in Fig. 1.

4.2 Test problems

This study incorporates a diverse set of constrained test problems gathered from the literature of structural optimization algorithms (Gandomi et al. 2011; Koziel and Yang 2011; Yang and Hossein Gandomi 2012; Jetton et al. 2023). With a focus on benchmarking the algorithm's ability to solve engineering problems, we gather six numerical test problems and nine engineering design optimization problems with both continuous and discrete value optimization. These 15 problems are detailed in Fig. 2 and “Appendix”.

4.3 Algorithm tests

The goal of the algorithm test is to provide a fair and comprehensive comparison of the state-of-the-art optimization methods. In this research, the optimization goal is to minimize the objective function for the test problems. The initial samplings for all test problems are performed with Latin Hypercube Sampling. Each test problem includes 50 sets of initial samples

that are randomly selected, with each initial set representing a separate experimental trial. In each individual experimental trial, all six algorithms begin the optimization process with the same set of initial samples. Furthermore, each algorithm is run for 200 iterations of optimization and timed for the run time for each experimental trial. The achieved optimal value and the total CPU run time of the 50 optimization trials for each test problem are then evaluated using our ranking procedure.

All algorithms are run on the same computer and Python environment to ensure the speed comparison is fair. The CPU is an Intel® Core™ i9-13900K Processor with 24 cores, and 128 GB of RAM. The system is GNU/Linux 6.5.0-15-generic x86_64 with Ubuntu 22.04.3 LTS as the operating system. While transformer models gain significantly from GPU acceleration and parallelization, we use only CPUs for a fairer comparison with CPU-run GPs.

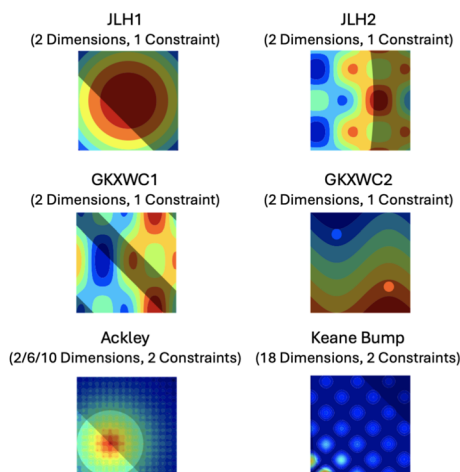
4.4 Evaluation metrics

4.4.1 Feasibility ratio

We define the optimization solution as the minimal value found by each method during optimization. However, for the BO algorithms utilized in this study, there is no guarantee of convergence to a feasible solution that respects all constraints. Therefore, the feasibility of the solution generated by each method is utilized as a metric for method evaluations. We define our constraint-handling performance evaluation methods as:

$$\text{Feasibility ratio} = \frac{\# \text{Trials with feasible solution}}{\text{Total \#trials} = 50}. \quad (10)$$

Numerical Problems:



Engineering Designs:

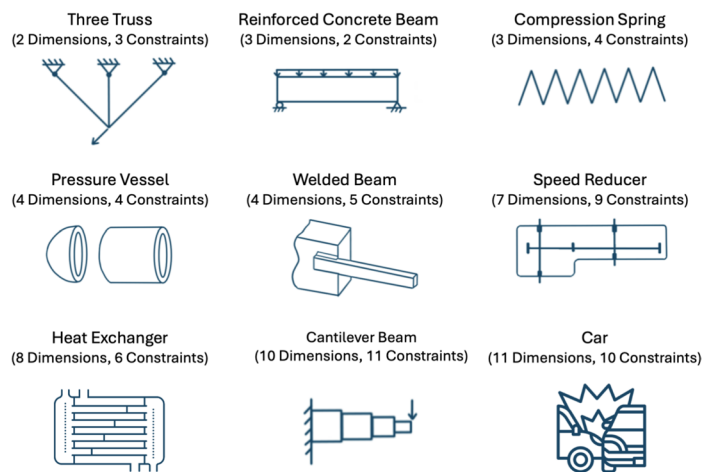


Fig. 2 Overview of the details of the 15 benchmark problems. The non-feasible regions are shaded in the numerical problems. The Ackley problem is experimented with optimization in 2D, 6D, and 10D, resulting in 17 experiment problems

4.4.2 Statistical ranking

In this study, we employed statistical ranking methods rather than raw performance values to analyze the overall performance of the benchmarked BO algorithms. This ranking approach effectively normalizes the scale differences between problems, as some optimization tasks naturally yield larger objective values than others. Additionally, rankings help mitigate the impact of outliers and extreme values that could otherwise skew the comparative analysis.

For the statistical analysis, we utilized the Friedman test and Wilcoxon signed-rank test with Holm's α correction. These statistical methods were chosen for their particular strengths in handling algorithm benchmarking data. As non-parametric tests, they make no assumptions about the underlying distribution of the performance metrics, which is essential since optimization results often exhibit non-normal distributions and contain outliers. The dependency structure in our experimental setup, where algorithms were tested using identical initial samples and seeds across the same set of experiment trails, is effectively addressed by these methods. The Wilcoxon signed-rank test is specifically designed for dependent paired comparisons, while the Friedman test accounts for the blocking effect of different test problems. Furthermore, when dealing with multiple algorithm comparisons, Holm's α correction helps control the family-wise error rate (Ismail Fawaz et al. 2019; Brockhoff and Tušar 2023; Picard and Ahmed 2024).

4.4.3 Fixed-budget analysis

Hansen et al. (2022) presented the concept of fixed-budget evaluations, a technique for comparing the efficiency of optimization algorithms by allotting specific computational resources for their execution. Our investigation employs two distinct fixed-budget analysis methodologies:

- (1) *Fixed-iteration approach* the performance of each optimization algorithm is evaluated after a pre-determined number of 200 iterations.
- (2) *Fixed-runtime approach* the performance outcomes of the algorithms are compared within an identical CPU time frame. In this study, the runtime budget is set to be the time required for the fastest method to execute 200 iterations.

5 Results

5.1 Feasibility ratio performance

The feasibility ratio quantifies the capability of identifying a useful solution within the constrained space after a fixed number of iterations. Table 1 presents the feasibility ratio for each method across the test problems. For most test problems, all methods successfully find a feasible solution. Yet, not all algorithms can find a feasible solution for the Ackley function (2D, 6D, 10D), GKXWC2, and the Heat Exchanger

Table 1 Feasibility ratio comparison across different constraint-handling Bayesian optimization methods for 17 test cases. The feasibility ratio represents the percentage of trials (out of 50) where each method successfully found solutions respecting all constraints, thus, indicating algorithmic robustness. Higher percentages indicate better

constraint-handling capabilities. The results highlight that PFN-based methods (particularly PFN-CEI and PFN-CEI+) consistently achieve superior feasibility rates, especially for challenging problems like Ackley 10D and Heat Exchanger, where traditional GP-based methods struggle

Test case	GP-Pen (%)	GP-CEI (%)	GP-CEI+ (%)	PFN-Pen (%)	PFN-CEI (%)	PFN-CEI+ (%)
JLH1	100	100	100	100	100	100
JLH2	100	100	100	100	100	100
GKXWC1	100	100	100	100	100	100
GKXWC2	92	100	100	100	100	100
Ackley 2D	98	100	100	100	100	100
Ackley 6D	100	98	98	100	100	100
Ackley 10D	32	86	78	92	94	92
Three Truss	100	100	100	100	100	100
Reinforced Concrete Beam	94	100	100	100	100	100
Compression Spring	100	100	100	100	100	100
Pressure Vessel	100	100	100	100	100	100
Welded Beam	100	100	100	100	100	100
Speed Reducer	100	100	100	100	100	100
Heat Exchanger	2	80	82	40	100	100
Cantilever Beam	100	100	100	100	100	100
Car	100	100	100	100	100	100
Keane Bump 18D	100	100	100	100	100	100

problem every time. In these more challenging problems, we note that algorithms employing CEI for constraint handling exhibit a higher feasibility ratio than those utilizing a penalty function. For instance, in the Ackley 10D problem, GP-Pen achieves 32% feasible results, while GP-CEI and GP-CEI+ reach 86% and 78% feasibility rates, respectively. This trend is even more evident in the Heat Exchanger example. The feasibility ratio for GP-based methods increases from 2 to approximately 80% with the implementation of CEI, and for PFN-based methods, it rises from 40 to 100% when switching from the penalty transform to CEI.

The feasibility ratio analysis also reveals that the simplest method, GP-Pen exhibits the lowest feasibility rate, as expected. For problems with relatively higher dimensions, such as Ackley 10D and Heat Exchanger, PFN-based constrained BO methods demonstrate a higher feasibility rate than GP-based methods overall.

5.2 Optimization performance at fixed iteration

Figure 3 displays the distribution of optimal and feasible solutions for each method across 17 problems of 200 iterations. Our analysis begins with the optimization performance of six CBO algorithms in different categories of optimization problems. For numerical problems such as Ackley 6D and 10D, GP-CEI+ is 60% and 68% better than PFN-CEI in optimization performance. However, note that these represent only the feasible samples, and Table 1 shows that GP-CEI+ only generates 78% feasible samples, while PFN-CEI generates 94% feasible samples for Ackley 10D. The Keane Bump 18D problem is known to be challenging for GP-based methods (Eriksson and Poloczek 2021), where the PFN-based methods surpass the GP-based methods by 10%.

For the nine engineering problems, PFN-based methods consistently rank highest compared to GP-based methods. The median solutions from the PFN-CEI method dominate

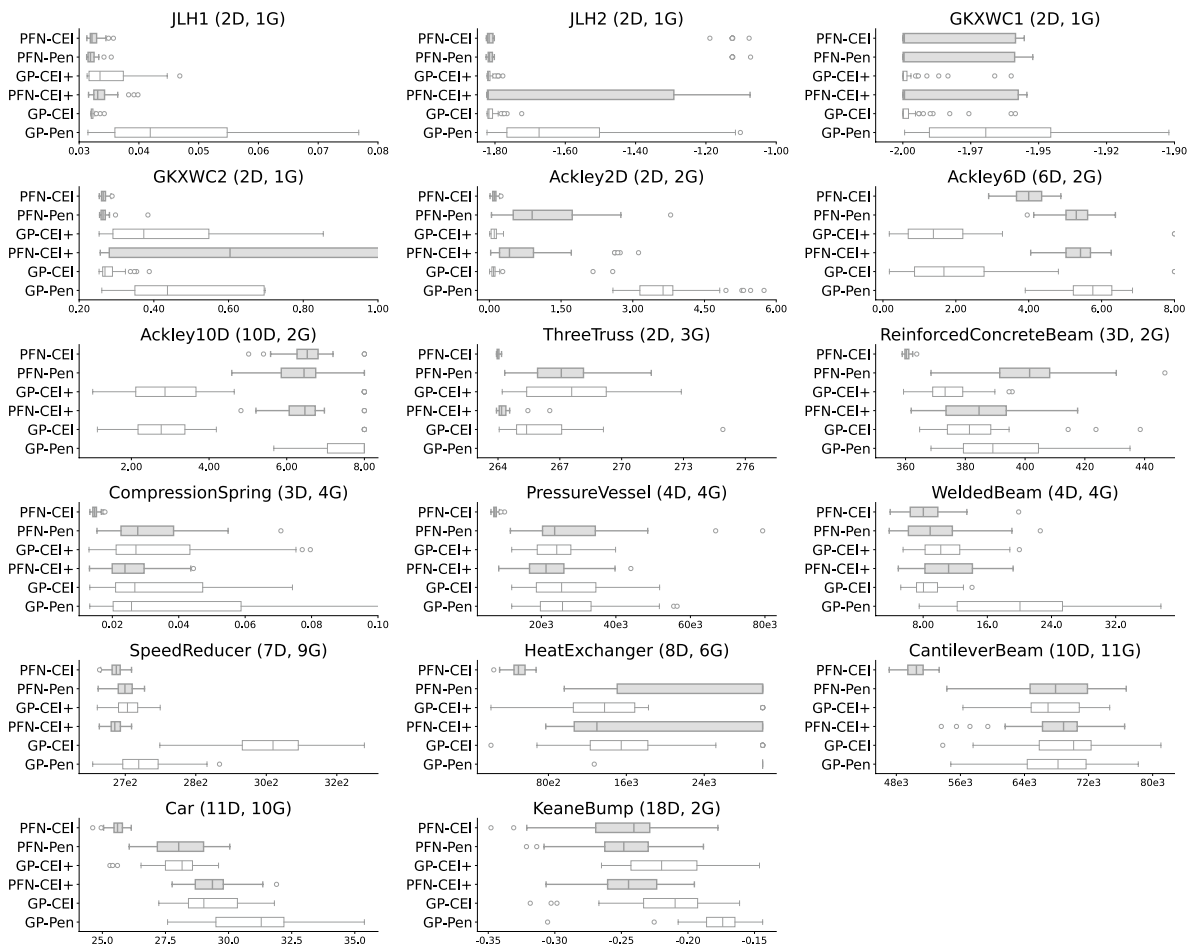


Fig. 3 Box plots comparing the optimal value achieved by each optimization method across 17 benchmark problems. The methods are ordered vertically from best to worst overall performance according to the statistical ranking shown in Fig. 6, with PFN-CEI (the best

performing method) at the top. Each box plot shows the distribution of optimal values found across 50 trials, where lower values indicate better optimization performance

all engineering problems, exhibiting performance two to three times better than that of GP-CEI or GP-CEI+.

5.3 Optimization performance at fixed-runtime

Figure 4 illustrates the convergence plot for each problem, highlighting the optimal value at a fixed time constraint marked by the completion of 200 iterations by PFN-Pen, the fastest approach. Upon the completion of PFN-Pen, a comparative analysis of performance outcomes reveals that PFN-CEI outperforms the others in 10 of the problems, while PFN-Pen leads in 6 cases, and PFN-CEI+ prevails in 1 case. PFN-based strategies consistently exhibit superior anytime performance throughout the operational timeframe defined by the termination of PFN-Pen.

Additionally, the convergence plot shows the advantage of PFN-based BO in limited runtime search, where GP-based BO sometimes cannot find feasible solutions. Specifically, for the Ackley 6D and 10D problems, although GP-based CEI methods outperform PFN-based methods after 200 iterations, the GP-based method is unable to find any constrained optimal solution within the given runtime limit. Even more evident, in the Compression Spring and Heat Exchanger problems, the PFN-based method outperforms in optimization, while GP-CEIs fail to find a feasible solution in the given time budget and perform worse than PFN-CEI at the fixed iteration.

5.4 Speed performance

In addition to the convergence plot in Fig. 4, Fig. 5 illustrates the Pareto trade-off between time and performance for each test problem. In both figures, algorithms using the penalty function are always faster than those utilizing CEI. PFN-Pen leads in speed on the Pareto front in all 17 benchmark problems, completing 200 iterations in 17.8 s on average. On the other hand, GP-Pen requires 36.5 s since GP is affected by the problems' expanding dimensions.

The speed disadvantage of GP's approach becomes more evident when comparing CEI methods for constraint handling. GP-CEI and GP-CEI+ need 647.9 and 590.8 s on average to perform 200 iterations of search, with a maximum of 6588.9 s (1.77 h) for running the Heat Exchanger problem. In contrast, PFN-CEI and PFN-CEI+ only need 55.8 s on average to finish an experiment, showing that they are 10 times faster than the GP-based CEI methods.

Moreover, CEI algorithm speeds drop as G goes from 1 to 11, with GP-based methods taking 430 times longer because of the feasibility calculations for each constraint. Due to the PFN's capability to solve both objectives and constraints in a single forward pass, PFN-based CEI methods' speeds only drop by a factor of 13, demonstrating the dominance of PFN-based methods in speed.

5.5 Overall rank

Figure 6 shows the critical difference plot from the statistical ranking of the six different CBO approaches. As stated in Sect. 4.4.2, a smaller rank (towards the left) indicates a better performance. The optimization performance critical difference plot result shows that left-most two PFN-based methods, PFN-CEI and PFN-Pen, outperform the traditional GP-based methods for the optimization performance rank. Surprisingly, applying CEI+ onto PFNs does not help improve the performance as it does for GP-CEI+. Furthermore, the time performance critical difference plot results show that the two methods using the penalty transform require the least time, with the PFN-Pen being the fastest method. The GP-CEI and GP-CEI+ methods are the slowest as expected.

6 Discussion

6.1 Recommendations for constrained Bayesian optimization methods

Overall, the results show that using PFN as BO's surrogate outperforms GP in speed and optimization performance. Notably, PFN algorithms, which are already 10 times faster than GP, could achieve even greater speeds through parallelization, as we tested all our models only on CPU. Considering the optimization performance, PFN-CEI achieves the highest ranking in optimization performance, followed by PFN-Pen. Although PFN-Pen completes 200 iterations more quickly, PFN-CEI achieves faster convergence, as shown in Fig. 4.

While PFN-based methods demonstrate both superior speed and optimization performance for most engineering problems, GP-based methods may still be preferred in specific scenarios. For numerical problems with complex landscapes like Ackley 6D and 10D, GP-CEI+ achieves better optimization performance despite being slower. Additionally, GP-based methods remain valuable when the problem dimension exceeds PFN's current 18D limitation and when the problem characteristics significantly deviate from PFN's training distribution.

Therefore, we recommend using PFN-CEI for overall performance and could do an early stop before 200 iterations if the user wants to perform optimization with a limited runtime budget. For specialized cases requiring high accuracy in numerical optimization or problem dimensions beyond 18D, GP-based methods remain a robust choice despite the speed trade-off.

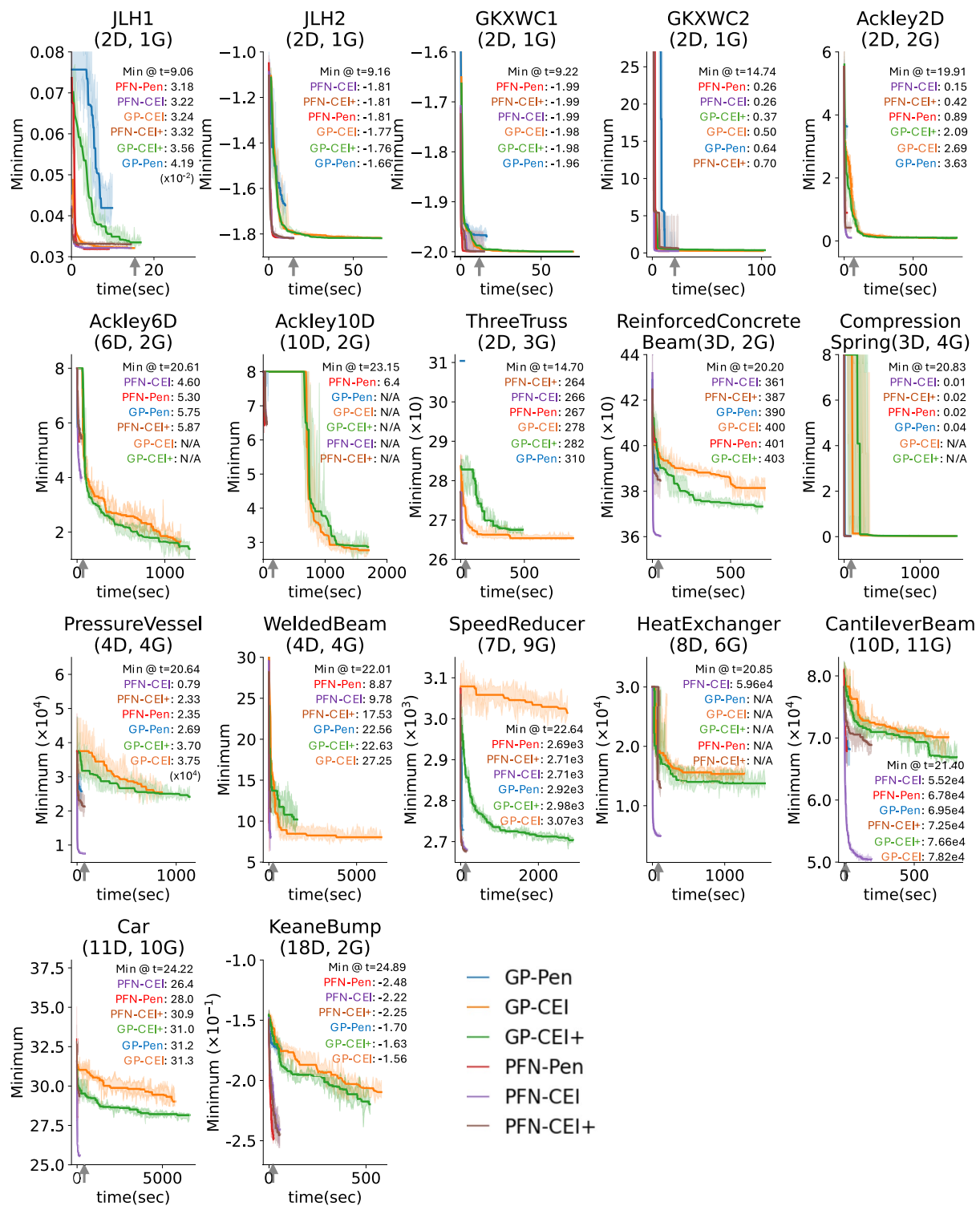


Fig. 4 Convergence plots comparing performance of optimization methods across all benchmark problems. Each line shows the median optimization trajectory over 50 trials, with shaded regions indicating 95% confidence intervals. The plots track the 200-iteration optimization

progress and each algorithm's runtime. For fixed-runtime analysis, the runtime limit is defined by the completion time of PFN-Pen (the fastest method) reaching 200 iterations, where the gray arrow is pointing to

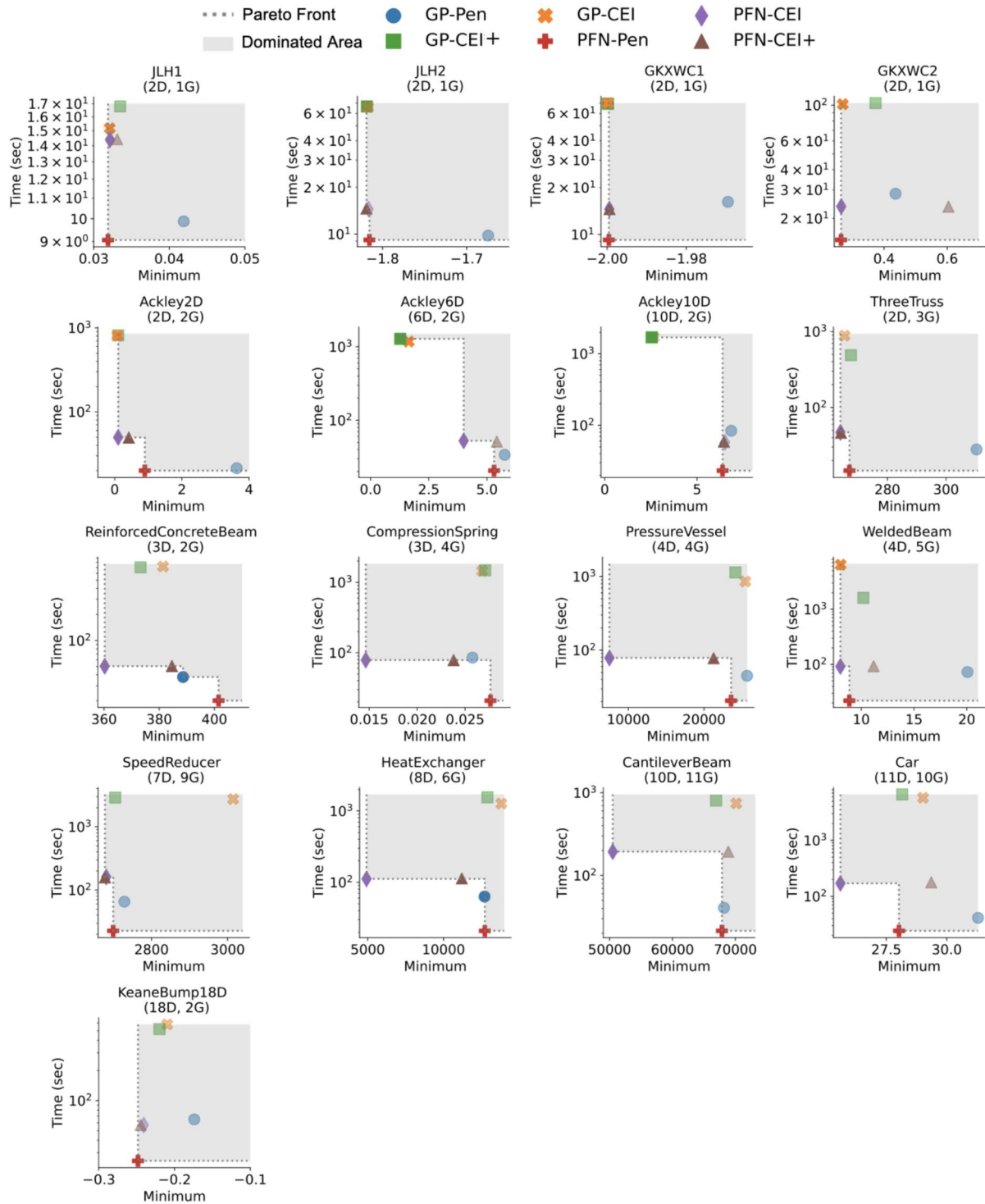


Fig. 5 Pareto plots demonstrating the trade-off between performance and total execution time (log-scale) for each method and test problem. D is the objective dimension, and G is the number of constraints. The average Pareto rank of each method over 17 experiment trials is [GP-

Pen, GP-CEI, GP-CEI+, PFN-Pen, PFN-CEI, PFN-CEI+] = [2.118, 2.353, 2.353, 1, 1.353, 1.765], where the smaller rank, the better, and rank 1 is the best. In problems with more than one constraint, PFN-based methods are 10 times faster than the GP-based CEI methods

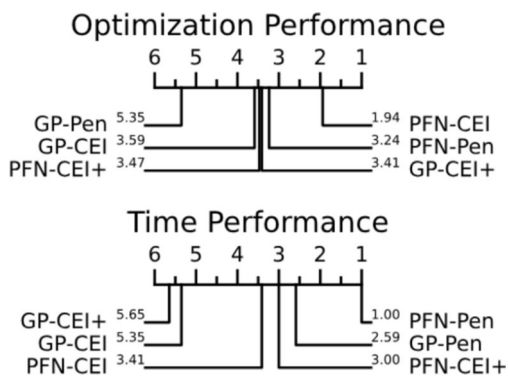


Fig. 6 Critical difference rank plot of overall results. A smaller rank indicates a better result. Regarding optimization performance, two PFN-based methods (PFN-CEI and PFN-Pen) lead. For time performance, PFN-Pen dominates as the fastest method, while the GP-based CEI ranks last

6.2 Potentials of pre-trained transformers-based BO

From the convergence plot and fixed-runtime analysis, it is evident that pre-trained-model-based BO is effective in rapidly assessing optimization problems and providing feasible solutions when GPs are unable to do so. The transformer architecture's impact on BO performance manifests through three key mechanisms. First, its attention mechanism enables parallel processing of multiple constraints and objectives, allowing efficient handling of complex optimization problems. Second, the architecture's ability to handle variable-sized inputs without retraining provides flexibility across different problem scales. Third, the efficient single-pass inference eliminates the need for iterative retraining, substantially reducing computational overhead.

Observing PFN's capability for rapid optimization through the transformer's ability to solve multiple functions in parallel in a single forward pass, we want to emphasize the potential of using pre-trained models for BO and its applications. Experimental optimization or user-guided BO (Jetton et al. 2024), which requires human input to the engineering optimization framework, will be time-sensitive as users must wait for BO to indicate the next potential optimum. The transformer-based approach's rapid processing capabilities are particularly valuable in these interactive scenarios, where minimizing latency is crucial for maintaining effective user engagement. Fast BO enables users to receive immediate feedback, enhancing the efficiency of the workflow.

Conversely, BO for hyperparameter tuning has been shown to take an extended period to identify the best hyperparameters for large models in image classification or language modeling (Cho et al. 2020). The parallel processing capabilities

and efficient inference characteristics of transformer-based BO could significantly accelerate this process, potentially revolutionizing the field of automated machine learning optimization. Transformer-based BO could potentially unlock the possibilities of fast hyperparameter optimization.

6.3 Understanding the complexity of test problems

One metric for comprehending the intricacy of the constrained test problems is to evaluate the feasible ratio of the six methods for each problem. Multi-modal numerical problems like Ackley, and problems with relatively small constrained areas, such as GKXWC2, are particularly challenging since not all methods have a 100% feasible ratio. With higher feasibility ratios in most engineering test cases, BO proves effective for constrained engineering design problem-solving. The Heat Exchanger problem, however, demonstrates the lowest overall feasibility among all problems with the longest runtime due to the exclusive presence of independent variables in the constraints and not in the objective function, making it the most complex engineering problem.

An alternative metric for evaluating problems is the variance in results across methods, as shown in Fig. 3. The choice of BO method is particularly vital for numerical problems, where results vary greatly, especially in cases like JHL2, GKXWC1, GKXWC2, and Ackley, due to their complexity. Engineering problems such as Pressure Vessel and Speed Reducer also have large variances in their results, underscoring the importance of method selection.

6.4 Limitations

This study provides insights into the application of PFN as black-box surrogates for BO while also acknowledging several inherent limitations.

6.4.1 Absence of acquisition function optimization

Firstly, the PFN-based methods employed in this research do not utilize acquisition function optimizers that are commonly implemented in BO algorithms. Theoretically, the absence of an acquisition optimizer could potentially accelerate the algorithm but suppress the optimization performance. While PFN-based BO still had the overall best performance, adding acquisition functions to them could further enhance their performance in multi-modal problems such as Ackley. Therefore, further evaluations of PFN-based BO utilizing standard acquisition optimization approaches are required.

6.4.2 Dimensionality and model scaling constraints

A key limitation of the current PFN-based methods is their restricted dimensionality capacity. However, the model can

only process up to 20 design features (# of objective + constraints), constraining its applicability to real-world engineering problems that often have higher dimensionalities. While transformer models like PFN can be scaled up to handle more dimensions, this requires substantial computational resources, taking approximately 24 h of training even with eight RTX 2080 Ti GPUs (Müller et al. 2023). Additionally, transformers are memory-intensive with quadratic dependency on input length, limiting processing to around 5000 training points on current consumer GPUs. Future research should explore efficient architectures that can handle higher-dimensional engineering design spaces while maintaining computational feasibility.

6.4.3 Limited validation in real engineering applications

Additionally, while PFNs show promise in addressing our nine engineering design problems through BO, this limited scope may not capture the full complexity and diversity encountered in practical engineering situations. We hope that our research will encourage other scholars to adopt PFN-based BO for their engineering design tasks and to evaluate novel algorithms using our benchmark problem sets. We also aim to expand the benchmark problem based on community input, providing a standardized test bed for research in constrained BO methods.

6.4.4 Constraints in performance evaluation framework

Lastly, our evaluation metrics were limited to runtime speed and optimization performance in fixed iterations. Future studies should explore additional aspects of Bayesian optimization, such as the convergence rate of iterations and scalability.

6.5 Future work

Our work in this paper lays the groundwork for several promising paths for future research. Recognizing that most engineering design problems involve multiple objectives, we aim to expand our constrained PFN-based BO methods to handle multi-objective optimization or active sampling by introducing additional acquisition functions or modifying the PFN architecture. Coupled with the dramatic speed improvements achieved by PFN-based methods, this expansion opens up opportunities for novel applications in time-sensitive domains. These include user-guided interactive BO for real-time design exploration, adaptive experiment design where quick decisions are crucial, and robotics control optimization requiring rapid adaptations. Additionally, PFN's efficiency could enable real-time hyperparameter tuning for large-scale machine learning models, addressing a significant bottleneck in model development.

Furthermore, we plan to benchmark our approach against more constraint BO methods such as Scalable Constrained Bayesian Optimization (SCBO; Eriksson and Poloczek 2021), as our current comparison is limited to CEI. This will involve a detailed comparison of PFN with other CBO methods using active sampling strategies or BO methods that employ neural networks instead of Gaussian Processes.

Another promising research direction involves tackling the challenge of high-dimensional optimization in Bayesian Optimization (BO). We aim to investigate the pre-training of larger transformer architectures tailored for higher-dimensional problems. Leveraging the rapid compute speed of PFNs, we also envision addressing high-dimensionality through strategies such as bootstrapping and aggregation, which hold significant potential for enhancing the performance of PFN models in this context.

Finally, we will continue expanding our benchmark suite based on community input, providing a standardized platform for evaluating new BO algorithms in engineering design. This expansion not only enhances the versatility of our approach but also opens up new possibilities for tackling complex optimization challenges in multidisciplinary optimization.

7 Conclusions

This research evaluates a novel approach for constraint-handling Bayesian optimization (CBO) by utilizing prior-data fitted networks (PFN) to remove the need for refitting the Gaussian Process (GP) for every searching iteration. Our comprehensive analysis is supported by benchmarking the methods on the 17 constrained optimization experiments, ranging from numerical synthesized test cases to engineering design problems. By using three constraint-handling approaches, penalty function PF, constrained expected improvement CEI, and modified constrained expected improvement CEI+, and two different surrogates, we evaluate six different CBO algorithms: GP-Pen, GP-CEI, GP-CEI+, PFN-Pen, PFN-CEI, PFN-CEI+. The results show that across the optimization problems, the PFN-based approach has dominated both performance and speed. PFN-CEI has the best optimization performance, followed by PFN-Pen and GP-CEI+, with exceptional performance in engineering problems. With the unique transformer architecture and pre-trained nature, PFN-based BO shows its capability to accelerate the BO process by an order of magnitude compared to GP-based Bayesian optimization.

Appendix: Benchmark problems

JLH1

This numerical problem is a two-dimensional “sphere” problem featuring a single optimum and a continuous inequality

constraint as detailed in (Jetton et al. 2023). The domain of interest for both x_1 and x_2 is $[0, 1]$.

$$f(x) = x_1^2 + x_2^2,$$

$$g(x) = x_1 + x_2 + 0.5 \leq 0.$$

JLH2

This numerical problem features a two-dimensional objective with local optimum used in multiple research (Jetton et al. 2024; Gardner et al. 2014). The continuous inequality constraint is proposed by Jetton et al. (2024). The domain of interest is $x_1 \in [-5, 0]$ and $x_2 \in [-5, 5]$.

$$f(x) = \cos(2x_1) \cos(x_2) + \sin(x_1),$$

$$g(x) = \frac{1}{4}(x_1 + 5)^2 + \frac{1}{100}x_2^2 - 1 \leq 0.$$

GKXWC1

The objective function of this question is identical to JHL2. However, the unique discontinuous inequality constraint is created by Gardner et al. (2014). The domain of interest for both x_1 and x_2 is $[0, 6]$.

$$f(x) = \cos(2x_1) \cos(x_2) + \sin(x_1),$$

$$g(x) = \cos(x_1) \cos(x_2) - \sin(x_1) \sin(x_2) - 0.5 \leq 0.$$

GKXWC2

This numerical problem is two-dimensional, featuring a multiple optimum and a discontinuous inequality constraint with a tiny feasible area proposed by Gardner et al. (2014). The domain of interest for both x_1 and x_2 is $[0, 6]$.

$$f(x) = \sin(x_1) + x_2,$$

$$g(x) = \sin(x_1) \sin(x_2) + 0.95 \leq 0.$$

Ackley function

Ackley function is a popular scalable numerical test case for optimization. It has many local minimums with the global optimal at $x = [0, 0, \dots]^n$ for a n -dimensional problem. We consider the domain of interest to be $[-5, 10]^n$ with $n = 2, 6, 10$. The constraints formulation is taken from Eriksen and Poloczec (2021). The large number of local minimum, multi-modal nature, and small feasible region makes this problem challenging.

$$f(x) = -20 \exp \left(-0.2 \sqrt{\frac{1}{n} \sum_{i=1}^n x_i^2} \right) - \exp \left(\frac{1}{n} \sum_{i=1}^n \cos(2\pi x_i) \right) + 20 + \exp(1),$$

$$g_1(x) = \sum_{i=1}^n x_i \leq 0,$$

$$g_2(x) = \|x\|_2 - 5 \leq 0.$$

Three truss

The objective of this problem is to minimize the volume of the three-bar truss while each truss is constrained by the stress acting on it. the given information are length (L) is 100 cm, pressure (P) is 2 kN/cm², and stress (σ) is 2 kN/cm². The domain of interest for both x_1 and x_2 is $[0, 1]$. The formulation is taken and varied from Yang and Hossein Gandomi (2012).

$$f(x) = (2\sqrt{2}x_1 + x_2)L,$$

$$g_1(x) = \frac{(\sqrt{2}x_1 + x_2)P}{\sqrt{2}x_1^2 + 2x_1x_2} - \sigma,$$

$$g_2(x) = \frac{x_2P}{\sqrt{2}x_1^2 + 2x_1x_2} - \sigma,$$

$$g_3(x) = \frac{P}{x_1 + \sqrt{2}x_2} - \sigma.$$

Reinforced concrete beam

This 3D problem describes a beam supported at two end points spaced by 30 ft. subjected to a live load and a dead load. The optimization objective is to minimize the cost while satisfying the constraints imposed by the loads and the safety requirement from ACI 318-77 code. This problem is representative of discrete value optimization. The cross-sectional area of the reinforcing bar (A_s) and the width of the concrete beam (b) are discrete while the depth of the concrete beam (h) is continuous. The domain of interest for each objective variable is: $A_s \in [0.2, 15]$ and $h \in [5, 10]$. As for b , it is the integers in $[28, 40]$. The formulation below is taken from Gandomi et al. (2011).

$$f(x) = 29.4A_s + 0.6bh,$$

$$g_1(x) = \frac{h}{b} - 4 \leq 0,$$

$$g_2(x) = 180 + \frac{7.35A_s^2}{b} - A_sh \leq 0.$$

Compression spring

This optimal spring design problem has many variations in the literature. Here we pick the helical compression spring design optimization problem proposed by Gandomi et al. (2011). The goal is to minimize the spring volume. The optimization variables are: the number of spring coils (N), the winding coil diameter (D), and the wire diameter (d). The domain of interest for each objective variable is $d \in [0.05, 1]$, $D \in [0.25, 1.3]$, and $N \in [1.5, 2]$.

$$\begin{aligned} f(x) &= (N + 2)Dd^2, \\ g_1(x) &= 1 - \frac{D^3N}{71,785d^4} \leq 0, \\ g_2(x) &= \frac{4D^2 - Dd}{12,566(Dd^3 - d^4)} + \frac{1}{5108d^2} - 1 \leq 0, \\ g_3(x) &= 1 - \frac{140.45d}{D^2N} \leq 0, \\ g_4(x) &= \frac{D + d}{1.5} - 1 \leq 0. \end{aligned}$$

Pressure vessel

The optimization of a cylindrical pressure vessel with both ends capped is to minimize the cost while meeting the ASME constraints on boilers and pressure vessels. The objective (cost) is defined with four optimization variables [thickness of the cylindrical skin (T_s), thickness of the spherical head (T_h), the inner radius (R), and length of the cylindrical segment of the vessel (L)]. The domain of interest for both T_s and T_h is $0.0625T$, where T is a random integer value from 1 to 99. For both R and L , the domain of interest is $[10, 200]$. The problem is originally proposed by Sandgren (1990) and the formulation is taken and varied from Gandomi et al. (2011).

$$\begin{aligned} f(x) &= 0.6224T_sRL + 1.7781T_hRR \\ &\quad + 3.1661T_sT_sL + 19.84T_sT_sR, \\ g_1(x) &= -T_s + 0.0193R \leq 0, \\ g_2(x) &= -T_h + 0.00954R \leq 0, \\ g_3(x) &= \pi R^2L - 4/3\pi R^3 + 1,296,000 \leq 0, \\ g_4(x) &= L - 240 \leq 0. \end{aligned}$$

Welded beam

The welded beam is designed to minimize the manufacturing cost. The five constraints are imposed on the shear stress, bending stress in the beam, geometry, buckling load on the beam, and deflection of the beam. The optimization variables are the thickness of the weld (h), the length of the welded joint (l), the width of the beam (t), and the thickness of the beam (b). The domain of interest is $h \in [0.125, 10]$, $l \in [0.1, 15]$, $t \in [0.1, 10]$, and $b \in [0.1, 10]$. Details about the problem formulation can be found in Gandomi et al. (2011).

$$\begin{aligned} f(x) &= 1.10471h^2l + 0.04811tb(14 + l), \\ \tau(x) &= \sqrt{\frac{\tau'(x)^2 + \tau''(x)^2 + l\tau'(x)\tau''(x)}{\sqrt{0.25(l^2 + (h + t)^2)}}}, \\ \tau'(x) &= \frac{6000}{\sqrt{2}hl}, \\ \tau''(x) &= \frac{6000(14 + 0.5l)\sqrt{0.25(l^2 + (h + t)^2)}}{2(0.707hl(\frac{l^2}{12} + 0.25(h + t)^2))}, \\ \sigma(x) &= \frac{504,000}{t^2b}, \\ P_c(x) &= 64,746(1 - 0.0282346t)tb^3, \\ \delta(x) &= \frac{2.1952}{t^3b}, \\ g_1(x) &= \tau(x) - 13,600, \leq 0 \\ g_2(x) &= \sigma(x) - 30,000, \leq 0 \\ g_3(x) &= b - h, \leq 0 \\ g_4(x) &= P_c - 6000, \leq 0 \\ g_5(x) &= 0.25 - \delta(x). \end{aligned}$$

Speed reducer

The speed reducer design is one of the most famous benchmark problems in structural engineering. The objective is to minimize the weight of the speed reducer while considering the dimensions of the inner bearings and shafts. The domain of interest is $b \in [2.6, 3.6]$, $m \in [0.7, 0.8]$, $z \in [17, 28]$, L_1 and $L_2 \in [7.3, 8.3]$, $d_1 \in [2.9, 3.9]$, and $d_2 \in [5, 5.5]$. The problem originated from Golinski (1973) and the formulation is taken from Yang and Hossein Gandomi (2012).

$$\begin{aligned}
f(x) &= 0.7854bm^2(3.3333z^2 + 14.9334z \\
&\quad - 43.0934) - 1.508b(d_1^2 + d_2^2) + 7.4777(d_1^3 + d_2^3) \\
&\quad + 0.7854(L_1d_1^2 + L_2d_2^2), \\
g_1(x) &= \frac{27}{bm^2z} - 1 \leq 0, \\
g_2(x) &= \frac{397.5}{bm^2z^2} - 1 \leq 0, \\
g_3(x) &= \frac{1.93L_1^3}{mzd_1^4} - 1 \leq 0, \\
g_4(x) &= \frac{1.93L_2^3}{mzd_2^4} - 1 \leq 0, \\
g_5(x) &= \frac{\sqrt{\left(\frac{745L_1}{mz}\right)^2 + 1.69 \times 10^6}}{110d_1^3} - 1 \leq 0, \\
g_6(x) &= \frac{\sqrt{\left(\frac{745L_2}{mz}\right)^2 + 157.5 \times 10^6}}{85d_2^3} - 1 \leq 0, \\
g_7(x) &= \frac{mz}{40} - 1 \leq 0, \\
g_8(x) &= \frac{5m}{b-1} - 1 \leq 0, \\
g_9(x) &= \frac{b}{12m} - 1 \leq 0.
\end{aligned}$$

Heat exchanger design

The Heat Exchanger design is known for its six complicated binding constraints. Although the objective is simple, satisfying all linear ($g_1 \sim g_3$) and nonlinear constraints ($g_4 \sim g_6$) make it a difficult constrained optimization problem. The domain of interest is $x_1 \in [10^2, 10^4]$, $x_{2 \sim 3} \in [10^3, 10^4]$, and $x_{4 \sim 8} \in [10, 10^3]$. Details of the problem formulation can be found here (Yang and Hossein Gandomi 2012).

$$\begin{aligned}
f(x) &= x_1 + x_2 + x_3, \\
g_1(x) &= 0.0025(x_4 + x_6) - 1 \leq 0, \\
g_2(x) &= 0.0025(x_5 + x_7 - x_4) - 1 \leq 0, \\
g_3(x) &= 0.01(x_8 - x_5) - 1 \leq 0, \\
g_4(x) &= 833.33252x_4 + 100x_1 - x_1x_6 \\
&\quad - 83,333.333 \leq 0, \\
g_5(x) &= 1250x_5 + x_2x_4 - x_2x_7 - 125x_4 \leq 0, \\
g_6(x) &= x_3x_5 - 2500x_5 - x_3x_8 + 1,250,000 \leq 0.
\end{aligned}$$

Cantilever stepped beam

This problem is originally proposed to minimize the volume of the stepped cantilever beam by Thanedar and

Vanderplaats (1995). In this 10D problem, we optimize a five-stepped cantilever beam with variable width x_i and height x_{i+5} of each step. The constant used are total length $L = 100$, Young's modulus $E = 2 \times 107$ GPa, and force applied on the end of the beam $P = 50,000$ N. The domain of interest for $x_{1 \sim 5}$ is $[1, 5]$ and for $x_{6 \sim 10}$ is $[30, 65]$. Details of the 11 constraints formulation can be found in Yang and Hossein Gandomi (2012) and Koziel and Yang (2011).

$$\begin{aligned}
f(x) &= \sum_{i=1}^5 x_i x_{i+5} l_i, \\
g_1(x) &= \frac{600P}{x_5 x_{10}^2} - 14,000 \leq 0, \\
g_2(x) &= \frac{6P(l_1 + l_2)}{x_4 x_9^2} - 14,000 \leq 0, \\
g_3(x) &= \frac{6P(l_1 + l_2 + l_3)}{x_3 x_8^2} - 14,000 \leq 0, \\
g_4(x) &= \frac{6P(l_1 + l_2 + l_3 + l_4)}{x_2 x_7^2} - 14,000 \leq 0, \\
g_5(x) &= \frac{6P(l_1 + l_2 + l_3 + l_4 + l_5)}{x_1 x_6^2} - 14,000 \leq 0, \\
g_6(x) &= \frac{PL^3}{3E} \left(\frac{1}{l_5} + \frac{7}{l_4} + \frac{19}{l_3} + \frac{37}{l_2} + \frac{61}{l_1} \right) - 2.7 \leq 0, \\
g_7(x) &= \frac{x_{10}}{x_5} - 20 \leq 0, \\
g_8(x) &= \frac{x_9}{x_4} - 20 \leq 0, \\
g_9(x) &= \frac{x_8}{x_3} - 20 \leq 0, \\
g_{10}(x) &= \frac{x_7}{x_2} - 20 \leq 0, \\
g_{11}(x) &= \frac{x_6}{x_1} - 20 \leq 0.
\end{aligned}$$

Car side impact design

The goal of this test case is to minimize the car weight while meeting the compatibility requirements for the car crash test. Eleven variables are used to describe the requirements imposed on the design, materials, and reinforcement of B-Pillar, floor side, cross members, door beam, door beltline, roof rail, barrier height, and impact position during the side impact test. This problem is originally proposed by Gu et al. (2001) and the formulation is taken from Gandomi et al. (2011). The domain for this test case is $x_{1,3 \sim 7} \in [0.5, 1.5]$, $x_2 \in [0.45, 1.35]$, $x_{8 \sim 9} \in [0.192, 0.345]$, and $x_{10 \sim 11} \in [-20, 0]$.

$$\begin{aligned}
f(x) &= 1.98 + 4.90x_1 + 6.67x_2 + 6.98x_3 \\
&\quad + 4.01x_4 + 1.78x_5 + 2.73x_7, \\
g_1(x) &= 1.16 - 0.3717x_2x_4 - 0.00931x_2x_{10} \\
&\quad - 0.484x_3x_9 + 0.01343x_6x_{10} - 1 \leq 0, \\
g_2(x) &= 0.261 - 0.0159x_1x_2 - 0.188x_1x_8 \\
&\quad - 0.019x_2x_7 + 0.0144x_3x_5 + 0.0008757x_5x_{10} \\
&\quad + 0.08045x_6x_9 + 0.00139x_8x_{11} + 0.00001575x_{10}x_{11} \\
&\quad - 0.9 \leq 0, \\
g_3(x) &= 0.214 + 0.00817x_5 - 0.131x_1x_8 - 0.0704x_1x_9 \\
&\quad + 0.03099x_2x_6 - 0.018x_2x_7 + 0.0208x_3x_8 + 0.121x_3x_9 \\
&\quad - 0.00364x_5x_6 + 0.0007715x_5x_{10} - 0.0005354x_6x_{10} \\
&\quad + 0.00121x_8x_{11} - 0.9 \leq 0, \\
g_4(x) &= 0.74 - 0.061x_2 - 0.163x_3x_8 + 0.001232x_3x_{10} \\
&\quad - 0.166x_7x_9 + 0.227x_2x_2 - 0.9 \leq 0, \\
g_5(x) &= 28.98 + 3.818x_3 - 4.2x_1x_2 + 0.0207x_5x_{10} \\
&\quad + 6.63x_6x_9 - 7.7x_7x_8 + 0.32x_9x_{10} - 32 \leq 0, \\
g_6(x) &= 33.86 + 2.95x_3 + 0.1792x_{10} - 5.057x_1x_2 \\
&\quad - 11.0x_2x_8 - 0.0215x_5x_{10} - 9.98x_7x_8 + 22.0x_8x_9 \\
&\quad - 32 \leq 0, \\
g_7(x) &= 46.36 - 9.9x_2 - 12.9x_1x_8 + 0.1107x_3x_{10} \\
&\quad - 32 \leq 0, \\
g_8(x) &= 4.72 - 0.5x_4 - 0.19x_2x_3 \\
&\quad - 0.0122x_4x_{10} + 0.009325x_6x_{10} + 0.000191x_{11}^2 \\
&\quad - 4 \leq 0, \\
g_9(x) &= 10.58 - 0.674x_1x_2 - 1.95x_2x_8 \\
&\quad + 0.02054x_3x_{10} - 0.0198x_4x_{10} + 0.028x_6x_{10} \\
&\quad - 9.9 \leq 0, \\
g_{10}(x) &= 16.45 - 0.489x_3x_7 - 0.843x_5x_6 \\
&\quad + 0.0432x_9x_{10} - 0.0556x_9x_{11} - 0.000786x_{11}^2 \\
&\quad - 15.7 \leq 0.
\end{aligned}$$

Keane bump

The Keane Bump benchmark is a scalable optimization test case proposed by Keane (1994) and it's known to be challenging for GP-based BO methods. The goal is to perform minimization over the domain $[0, 10]^d$. Here we use $d = 18$ since this is the maximum dimension that can be passed into the model used for PFNs4BO (Müller et al. 2023).

$$f(x) = - \left| \frac{\sum_{i=1}^d \cos^4(x_i) - 2 \prod_{i=1}^d \cos^2(x_i)}{\sqrt{\sum_{i=1}^d ix_i^2}} \right|,$$

$$g_1(x) = 0.75 - \prod_{i=1}^d x_i \leq 0,$$

$$g_2(x) = \sum_{i=1}^d x_i - 7.5d \leq 0.$$

Acknowledgements The authors would like to acknowledge the MIT-IBM Watson AI Lab in supporting this research.

Funding 'Open Access funding provided by the MIT Libraries'. Cyril Picard: Swiss National Science Foundation (Postdoc.Mobility Fellowship P500PT_{2}06937).

Declarations

Conflict of interest On behalf of all authors, the corresponding author states that there is no conflict of interest.

Replication of results This paper provides detailed descriptions of the benchmark example in the "Appendix" to support result replication. The Python code for the benchmarks and GP- and PFN-based BO can be found in this GitHub repo <https://github.com/rosenyu304/BOEngineeringBenchmark> with tutorials for the use of each algorithms provided. The GPs are built with the BoTorch library, and the PFN model is taken from Müller et al. (2023).

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by/4.0/>.

References

- Alarie S, Audet C, Gheribi AE, Kokkolaras M, Le Digabel S (2021) Two decades of black box optimization applications. *EURO J Comput Optim* 9:100011
- Ali A, Hamraz M, Kumam P, Khan DM, Khalil U, Sulaiman M, Khan Z (2020) A k-nearest neighbours based ensemble via optimal model selection for regression. *IEEE Access* 8:132095–132105

- Bagheri S, Konen W, Allmendinger R, Branke J, Deb K, Fieldsend J, Quagliarella D, Sindhya K (2017) Constraint handling in efficient global optimization. In: Proceedings of the genetic and evolutionary computation conference, 2017, pp 673–680
- Bajaj I, Arora A, Hasan MF (2021) Black-box optimization: methods and applications. In: Black box optimization, machine learning, and no-free lunch theorems. Springer, Berlin, pp 35–65
- Balandat M, Karrer B, Jiang D, Daulton S, Letham B, Wilson AG, Bakshy E (2020) BoTorch: a framework for efficient Monte Carlo Bayesian optimization. In: Advances in neural information processing systems, 2020, vol 33. <https://proceedings.neurips.cc/paper/2020/hash/f5b1b89d98b7286673128a5fb112cb9a-Abstr.html>
- Baptista R, Poloczek M (2018) Bayesian optimization of combinatorial structures. In: International conference on machine learning, 2018. PMLR, pp 462–471
- Biswas A, Hoyle C (2021) An approach to Bayesian optimization for design feasibility check on discontinuous black-box functions. *J Mech Des* 143(3):031716
- Blank J, Deb K (2020) Pymoo: multi-objective optimization in Python. *IEEE Access* 8:89497–89509
- Brockhoff D, Tušar T (2023) GECCO 2023 tutorial on benchmarking multiobjective optimizers 2.0. In: Proceedings of the companion conference on genetic and evolutionary computation, 2023, pp 1183–1212
- Cardoso I, Dubreuil S, Bartoli N, Gogu C, Salaün M (2024) Constrained efficient global multidisciplinary design optimization using adaptive disciplinary surrogate enrichment. *Struct Multidisc Optim* 67(2):23
- Cho H, Kim Y, Lee E, Choi D, Lee Y, Rhee W (2020) Basic enhancement strategies when using Bayesian optimization for hyperparameter tuning of deep neural networks. *IEEE Access* 8:52588–52608
- Cowen-Rivers AI, Lyu W, Tutunov R, Wang Z, Grosnit A, Griffiths RR, Maraval AM, Jianye H, Wang J, Peters J, Bou-Amman H (2022) HEBO: pushing the limits of sample-efficient hyperparameter optimisation. *J Artif Intell Res* 74:1269–1349
- Cunningham JP, Shenoy KV, Sahani M (2008) Fast Gaussian process methods for point process intensity estimation. In: Proceedings of the 25th international conference on machine learning, 2008, pp 192–199
- Du X, Liang J, Lei J, Xu J, Xie P (2023) A radial-basis function mesh morphing and Bayesian optimization framework for vehicle crashworthiness design. *Struct Multidisc Optim* 66(3):64
- Eriksson D, Poloczek M (2021) Scalable constrained Bayesian optimization. In: International conference on artificial intelligence and statistics, 2021. PMLR, pp 730–738
- Fletcher R (1975) An ideal penalty function for constrained optimization. *IMA J Appl Math* 15(3):319–342
- Foreman-Mackey D, Agol E, Ambikasaran S, Angus R (2017) Fast and scalable Gaussian process modeling with applications to astronomical time series. *Astron J* 154(6):220
- Gandomi AH, Yang XS, Alavi AH (2011) Mixed variable structural optimization using firefly algorithm. *Comput Struct* 89(23–24):2325–2336
- Gardner JR, Kusner MJ, Xu ZE, Weinberger KQ, Cunningham JP (2014) Bayesian optimization with inequality constraints. In: ICML, 2014, pp 937–945
- Gardner J, Guo C, Weinberger K, Garnett R, Grosse R (2017) Discovering and exploiting additive structure for Bayesian optimization. In: Artificial intelligence and statistics, 2017. PMLR, pp 1311–1319
- Garnett R (2015) Lecture 12 Bayesian optimization. Lecture notes in CSE515T: Bayesian methods in machine learning
- Garnett R (2023) Bayesian optimization. Cambridge University Press, Cambridge
- Gelbart MA, Snoek J, Adams RP (2014) Bayesian optimization with unknown constraints. *arXiv preprint*. [arXiv:1403.5607](https://arxiv.org/abs/1403.5607)
- Ghoreishi SF, Allaire D (2019) Multi-information source constrained Bayesian optimization. *Struct Multidisc Optim* 59:977–991
- Gilboa E, Saatçi Y, Cunningham JP (2013) Scaling multidimensional inference for structured Gaussian processes. *IEEE Trans Pattern Anal Mach Intell* 37(2):424–436
- Golinski J (1973) An adaptive optimization system applied to machine synthesis. *Mech Mach Theory* 8(4):419–436
- Greenhill S, Rana S, Gupta S, Vellanki P, Venkatesh S (2020) Bayesian optimization for adaptive experimental design: a review. *IEEE Access* 8:13937–13948
- Gu L, Yang RJ, Tho CH, Makowskit M, Faruquet O, Y. Li YL (2001) Optimisation and robustness for crashworthiness of side impact. *Int J Veh Des* 26(4):348–360
- Hansen N, Auger A, Ros R, Mersmann O, Tušar T, Brockhoff D (2021) COCO: a platform for comparing continuous optimizers in a black-box setting. *Optim Methods Softw* 36(1):114–144
- Hansen N, Auger A, Brockhoff D, Tušar T (2022) Anytime performance assessment in black-box optimization benchmarking. *IEEE Trans Evol Comput* 26(6):1293–1305
- Ismail Fawaz H, Forestier G, Weber J, Idoumghar L, Muller PA (2019) Deep learning for time series classification: a review. *Data Min Knowl Discov* 33(4):917–963
- Jetton C, Li C, Hoyle C (2023) Constrained Bayesian optimization methods using regression and classification Gaussian processes as constraints. In: International design engineering technical conferences and computers and information in engineering conference, 2023. American Society of Mechanical Engineers, p V03BT03A033
- Jetton C, Campbell M, Hoyle C (2024) Constraining the feasible design space in Bayesian optimization with user feedback. *J Mech Des* 146:041703–1
- Kamrah E, Ghoreishi SF, Ding ZJ, Chan J, Fuge M (2023) How diverse initial samples help and hurt Bayesian optimizers. *J Mech Des* 145(11):111703
- Keane A (1994) Experiences with optimizers in structural design. In: Proceedings of the conference on adaptive computing in engineering design and control, 1994, pp 14–27
- Khan NA, Sulaiman M, Aljohani AJ, Kumam P, Alrabaiah H (2020) Analysis of multi-phase flow through porous media for imbibition phenomena by using the LeNN-WOA-NM algorithm. *IEEE Access* 8:196425–196458
- Khan NA, Sulaiman M, Aljohani AJ, Bakar MA (2022) Mathematical models of CBSC over wireless channels and their analysis by using the LeNN-WOA-NM algorithm. *Eng Appl Artif Intell* 107:104537
- Klein A, Falkner S, Bartels S, Hennig P, Hutter F (2017) Fast Bayesian optimization of machine learning hyperparameters on large datasets. In: Artificial intelligence and statistics, 2017. PMLR, pp 528–536
- Koziel S, Yang XS (2011) Computational optimization, methods and algorithms, vol 356. Springer, Berlin
- Liang Q, Gongora AE, Ren Z, Tihihonen A, Liu Z, Sun S, Deneault JR, Bash D, Mekki-Berrada F, Khan SA, Hippalgaonkar K (2021) Benchmarking the performance of Bayesian optimization across multiple experimental materials science domains. *NPJ Comput Mater* 7(1):188
- Liu H, Ong YS, Shen X, Cai J (2020) When Gaussian process meets big data: a review of scalable GPS. *IEEE Trans Neural Netw Learn Syst* 31(11):4405–4423
- Martinez-Cantin R (2018) Funneled Bayesian optimization for design, tuning and control of autonomous systems. *IEEE Trans Cybern* 49(4):1489–1500
- Mathern A, Steinholtz OS, Sjöberg A, Önnheim M, Ek K, Rempling R, Gustavsson E, Jirstrand M (2021) Multi-objective constrained

- Bayesian optimization for structural design. *Struct Multidisc Optim* 63:689–701
- Müller S, Hollmann N, Arango SP, Grabocka J, Hutter F (2021) Transformers can do Bayesian inference. *arXiv preprint*. [arXiv:2112.10510](https://arxiv.org/abs/2112.10510)
- Müller S, Feurer M, Hollmann N, Hutter F (2023) PFNs4BO: in-context learning for Bayesian optimization. *arXiv preprint*. [arXiv:2305.17535](https://arxiv.org/abs/2305.17535)
- Picard C, Ahmed F (2024) Fast and accurate zero-training classification for tabular engineering data. *arXiv preprint*. [arXiv:2401.06948](https://arxiv.org/abs/2401.06948)
- Picard C, Schiffmann J (2021) Realistic constrained multiobjective optimization benchmark problems from design. *IEEE Trans Evol Comput* 25(2):234–246. <https://doi.org/10.1109/TEVC.2020.3020046>
- Pleiss G, Jankowiak M, Eriksson D, Damle A, Gardner J (2020) Fast matrix square roots with applications to Gaussian processes and Bayesian optimization. *Adv Neural Inf Process Syst* 33:22268–22281
- Ragueneau Q, Laurent L, Legay A, Larroque T, Crambuer R (2024) A constrained Bayesian optimization framework for structural vibrations with local nonlinearities. *Struct Multidisc Optim* 67(4):1–27
- Sandgren E (1990) Nonlinear integer and discrete programming in mechanical design optimization. *J Mech Des* 112(2):223–229. <https://doi.org/10.1115/1.2912596>
- Shahriari B, Swersky K, Wang Z, Adams RP, De Freitas N (2015) Taking the human out of the loop: a review of Bayesian optimization. *Proc IEEE* 104(1):148–175
- Shields BJ, Stevens J, Li J, Parasram M, Damani F, Alvarado JI, Janey JM, Adams RP, Doyle AG (2021) Bayesian reaction optimization as a tool for chemical synthesis. *Nature* 590(7844):89–96
- Tao S, Van Beek A, Apley DW, Chen W (2021) Multi-model Bayesian optimization for simulation-based design. *J Mech Des* 143(11):111701
- Thanedar P, Vanderplaats G (1995) Survey of discrete variable optimization for structural design. *J Struct Eng* 121(2):301–306
- Tran A, Tran M, Wang Y (2019) Constrained mixed-integer Gaussian mixture Bayesian optimization and its applications in designing fractal and auxetic metamaterials. *Struct Multidisc Optim* 59:2131–2154
- Tran A, Eldred M, Wildey T, McCann S, Sun J, Visintainer RJ (2022) aphBO-2GP-3B: a budgeted asynchronous parallel multi-acquisition functions for constrained Bayesian optimization on high-performing computing architecture. *Struct Multidisc Optim* 65(4):132
- Wang GG, Shan S (2006) Review of metamodeling techniques in support of engineering design optimization. In: *International design engineering technical conferences and computers and information in engineering conference*, 2006, pp 415–426
- Yang XS, Hossein Gandomi A (2012) Bat algorithm: a novel approach for global engineering optimization. *Eng Comput* 29(5):464–483
- Zhang Y, Lin J, Hu Z, Khan NA, Sulaiman M (2021) Analysis of third-order nonlinear multi-singular Emden–Fowler equation by using the LeNN-WOA-NM algorithm. *IEEE Access* 9:72111–72138

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.